libatrac3plus Reference

© 2009 Sony Computer Entertainment Inc.
All Rights Reserved.
SCE Confidential

Table of Contents

Structure **4**
SceAtracBufferInfo 5
SceAtracLowLevelParam 7

ATRAC™ ID Management Functions **8**
sceAtracGetAtracID 9
sceAtracReleaseAtracID 11
sceAtracReinit 13

Initialization Function **15**
sceAtracSetData 16
sceAtracSetMOutData 18
sceAtracSetHalfwayBuffer 20
sceAtracSetMOutHalfwayBuffer 22
sceAtracSetDataAndGetID 24
sceAtracSetAA3DataAndGetID 26
sceAtracSetMOutDataAndGetID 28
sceAtracSetHalfwayBufferAndGetID 30
sceAtracSetAA3HalfwayBufferAndGetID 32
sceAtracSetMOutHalfwayBufferAndGetID 34

Decode Function **36**
sceAtracDecodeData 37

Streaming and Buffer-Related Functions **40**
sceAtracGetStreamDataInfo 41
sceAtracAddStreamData 43
sceAtracGetSecondBufferInfo 45
sceAtracSetSecondBuffer 47
sceAtracIsSecondBufferNeeded 49
sceAtracGetRemainFrame 51

Audio Information Acquisition Functions **53**
sceAtracGetChannel 54

sceAtracGetOutputChannel 56
sceAtracGetMaxSample 58
sceAtracGetNextSample 60
sceAtracGetSoundSample 62
sceAtracGetNextDecodePosition 65
sceAtracGetBitrate 67
sceAtracGetLoopStatus 69

Loop Count Setting Function 71

sceAtracSetLoopNum 72

Playback Position Resetting Functions 74

sceAtracGetBufferInfoForResetting 75
sceAtracResetPlayPosition 77

Low-Level Functions 79

sceAtracLowLevelInitDecoder 80
sceAtracLowLevelDecode 82

Detailed Error Acquisition Function 84

sceAtracGetInternalErrorInfo 85

Constant Definitions 87

Codec Identifier 88
Buffer State Identifier 89

Error Codes 91

List of Error Codes 92

Structure

SceAtracBufferInfo

Structure for buffer control information when resetting playback position

Structure

```
#include <libatrac3plus>

typedef struct {
        SceUChar8 *pucWritePositionFirstBuf;
        SceUInt32 uiWritableByteFirstBuf;
        SceUInt32 uiMinWriteByteFirstBuf;
        SceUInt32 uiReadPositionFirstBuf;
        SceUChar8 *pucWritePositionSecondBuf;
        SceUInt32 uiWritableByteSecondBuf;
        SceUInt32 uiMinWriteByteSecondBuf;
        SceUInt32 uiReadPositionSecondBuf;
} SceAtracBufferInfo;
```

Members

**pucWritePositionFirstBuf*	Start address for writing to first buffer
uiWritableByteFirstBuf	Maximum number of bytes which can be written to first buffer
uiMinWriteByteFirstBuf	Minimum number of bytes which must be written to first buffer
uiReadPositionFirstBuf	Read position of data file to write to first buffer
**pucWritePositionSecondBuf*	Start address for writing to second buffer
uiWritableByteSecondBuf	Maximum number of bytes which can be written to second buffer
uiMinWriteByteSecondBuf	Minimum number of bytes which must be written to second buffer
uiReadPositionSecondBuf	Read position of data file to write to second buffer

Description

This structure is used for obtaining buffer control data when resetting the playback position.

See Also

sceAtracGetBufferInfoForResetting()

sceAtracResetPlayPosition()

SceAtracLowLevelParam

Low-level decoder parameter structure

Structure

```
#include <libatrac3plus.h>

typedef struct {
        SceUInt32 nchin;
        SceUInt32 nchout;
        SceUInt32 nFrameBytes;
} SceAtracLowLevelParam;
```

Members

nchin	Number of input channels
nchout	Number of output channels
nFrameBytes	Size of one frame of ATRAC3plus™ data

Description

This structure is used by **sceAtracLowLevelInitDecoder**() for initializing the ATRAC™ ID.

See Also

sceAtracLowLevelInitDecoder()

ATRAC™ ID Management Functions

sceAtracGetAtracID

Get an ATRAC™ ID

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracGetAtracID(
        SceUInt32 uiCodecType
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or -enabled state)

Multithread safe

Arguments

uiCodecType Identifier indicating type of AudioCodec to be used

Return Value

Value	Meaning
0	Normal completion
< 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function allocates an ATRAC™ ID. An ATRAC™ ID must be allocated before ATRAC3™/ATRAC3plus™ data can be decoded.

Example

```
/* Allocate an ATRAC™ ID for ATRAC3™ data*/
atracId = sceAtracGetAtracID(SCE_ATRAC_AT3);
```

Notes

The user must call sceAtracGetAtracID() specifying the proper codec to be used to obtain the ATRAC™ ID.

The ATRAC™ ID that was allocated by specifying SCE_ATRAC_AT3 for *uiCodecType* cannot be used for ATRAC3plus™ data, and the ATRAC™ ID that was allocated by specifying SCE_ATRAC_AT3PLUS for *uiCodecType* cannot be used for ATRAC3™ data

See Also

Codec Identifier

sceAtracReleaseAtracID()

sceAtracReleaseAtracID

Release an ATRAC™ ID

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracReleaseAtracID(
        SceUInt32 atracID
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or -enabled state)

Multithread safe

Arguments

atracID ID for managing the decoding of ATRAC3™/ATRAC3plus™ data

Return Value

Value	Meaning
0	Normal completion
< 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function releases an ATRAC™ ID.

Example

```
/* Release an ATRAC™ ID */
status = sceAtracReleaseAtracID(atracID);
```

Notes

The number of ATRAC™ IDs that can be used simultaneously is limited. Please release an ATRAC™ ID after it is through being used, otherwise it may not be possible for another thread that is running concurrently to allocate one.

See Also

Codec Identifier

sceAtracGetAtracID()

sceAtracReinit

Reinitialize ATRAC™ ID

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracReinit(
        int     at3_entry,
        int     at3plus_entry
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or -enabled state)

Multithread safe

Arguments

at3_entry	Number of ATRAC3™ ATRAC™ IDs to be used simultaneously
at3plus_entry	Number of ATRAC3plus™ ATRAC™ IDs to be used simultaneously

Return Value

Value	Meaning
0	Normal completion
<0	Error

For return values when an error occurs, see the list of error codes at the end of this document.

Description

This function reinitializes the library and changes the number of ATRAC™ IDs that can be used simultaneously.

By default, the number of ATRAC™ IDs that can be used simultaneously is 2 for ATAC3 and 2 for ATRAC3plus™. This function can be used to change these limits in a range that satisfies the following relationship.

(Number of ATRAC3™ IDs) + (Number of ATRAC3plus™ IDs) x 2 <= 6

If the above condition is not satisfied, an error is returned.

All ATRAC™ IDs must be released before this function is called.

Example

```
/* Reinitialize by setting four ATRAC3™ IDs and one ATRAC3plus™ ID */
res = sceAtracReinit(4,1);
```

See Also

sceAtracGetAtracID()

sceAtracReleaseAtracID()

Initialization Function

sceAtracSetData

Initial data setting function

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracSetData(
        SceInt32 atracID,
        SceUChar8 *pucBufferAddr,
        SceUInt32 uiBufferByte
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (must be called in an interrupt-enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
pucBufferAddr	Starting address of input data
uiBufferByte	Size of input buffer

Return Value

Value	Meaning
0	Normal completion
< 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function associates an ATRAC™ ID with an input buffer and registers it in the library. The ATRAC™ ID must be allocated before this function is called. Also, data from the beginning of the file should be read into the input buffer.
This function accepts monaural and stereo data as input data. If the decoding function **sceAtracDecodeData**() is called after initialization is performed by this function, the data will always be output as two channels.
By this function, memory required by libatrac3plus is initialized, the buffer that was configured as *pucBufferAddr* is set, and the header of the ATRAC3™/ATRAC3plus™ data that was read into *pucBufferAddr* is analyzed.

Example

```
/* Associate ATRAC™ ID and input buffer.*/
status = sceAtracSetData(atracID, buf, databyte);
```

Notes

If data is not read correctly to the input buffer, up to *uiBufferByte* + 2 bytes from *pucBufferAddr* may be accessed.
Therefore, make sure that the 2 bytes after the final address of the read buffer are located in an accessible area.

sceAtracSetMOutData

Initial data setting function (monaural output)

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracSetMOutData(
        SceInt32 atracID,
        SceUChar8 *pucBufferAddr,
        SceUInt32 uiBufferByte
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (must be called in an interrupt-enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
pucBufferAddr	Starting address of input data
uiBufferByte	Input buffer size

Return Value

Value	Meaning
0	Normal completion
< 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function associates an ATRAC™ ID with an input buffer and registers it in the library. Data from the beginning of the file should be read into the input buffer. The ATRAC™ ID must be allocated before this function is called.

This function only accepts monaural data as input data. If the decoding function **sceAtracDecodeData**() is called after initialization is performed by this function, the output data will always be one channel.

Internally, memory required by libatrac3plus is initialized, the buffer that was configured as *pucBufferAddr* is set, and the header of the ATRAC3™/ATRAC3plus™ data that was read into *pucBufferAddr* is analyzed.

Example

```
/* Associate an ATRAC™ ID with an input buffer */
status = sceAtracSetMOutData(atracID, buf, databyte);
```

Notes

If data is not read correctly into the input buffer, up to *uiBufferByte* + 2 bytes from *pucBufferAddr* may be accessed.

Therefore, make sure that the 2 bytes after the final address of the read buffer are located in an accessible area.

sceAtracSetHalfwayBuffer

Initial data setting function

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracSetHalfwayBuffer(
        SceInt32 atracID,
        SceUChar8 *pucBufferAddr,
        SceUInt32 uiReadByte,
        SceUInt32 uiBufferByte
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (must be called in interrupt-enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
pucBufferAddr	Starting address of input data
uiReadByte	Size of valid data from beginning of input data
uiBufferByte	Input buffer size

Return Value

Value	Meaning
0	Normal completion
< 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function associates an ATRAC™ ID with an input buffer and registers it in the library. The ATRAC™ ID must be allocated before this function is called. Also, data from the beginning of the file should be read into the input buffer.
This function accepts monaural and stereo data as input data. If the decoding function **sceAtracDecodeData**() is called after initialization is performed by this function, the data will always be output as two channels.
Internally, memory required by libatrac3plus is initialized, the buffer that was configured as *pucBufferAddr* is set, and the header of the ATRAC3™/ATRAC3plus™ data that was read into *pucBufferAddr* is analyzed.
This function differs from **sceAtracSetData**() in that the amount of data that is read into the buffer can be less than the buffer size. Therefore, the time until sound is first produced can be shortened since the amount of data that must be read up front can be reduced.

Example

```
/* Associate an ATRAC™ ID with an input buffer */
status = sceAtracSetHalfwayBuffer(atracID, buf, readbyte,bufsize);
```

Notes

If data is not read correctly to the input buffer, up to *uiBufferByte* + 2 bytes from *pucBufferAddr* may be accessed.
Therefore, make sure that the 2 bytes after the final address of the read buffer are located in an accessible area.

sceAtracSetMOutHalfwayBuffer

Initial data setting function (monaural output)

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracSetMOutHalfwayBuffer(
        SceInt32 atracID,
        SceUChar8 *pucBufferAddr,
        SceUInt32 uiReadByte,
        SceUInt32 uiBufferByte
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (must be called in an interrupt-enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
pucBufferAddr	Starting address of input data
uiReadByte	Size of valid data from beginning of input data
uiBufferByte	Input buffer size

Return Value

Value	Meaning
0	Normal completion
< 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function associates an ATRAC™ ID with an input buffer and registers it in the library. The ATRAC™ ID must be allocated before this function is called. Also, data from the beginning of the file should be read into the input buffer.
This function accepts only monaural data as input data. If the decoding function **sceAtracDecodeData**() is called after initialization is performed by this function, the data will always be output as one channel.
Internally, memory required by libatrac3plus is initialized, the buffer that was configured as *pucBufferAddr* is set, and the header of the ATRAC3™/ATRAC3plus™ data that was read into *pucBufferAddr* is analyzed.
This function differs from **sceAtracSetMOutData**() in that the amount of data that is read into the buffer can be less than the buffer size. Therefore, the time until sound is first produced can be shortened since the amount of data that must be read up front can be reduced.

Example

```
/* Associate an ATRAC™ ID with an input buffer */
status = sceAtracSetMOutHalfwayBuffer(atracID, buf, readbyte,bufsize);
```

Notes

If data is not read correctly into the input buffer, up to *uiBufferByte* + 2 bytes from *pucBufferAddr* may be accessed.
Therefore, make sure that the 2 bytes after the final address of the read buffer are located in an accessible area.

sceAtracSetDataAndGetID

Set initial data and allocate ATRAC™ ID

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracSetDataAndGetID(
        SceUChar8 *pucBufferAddr,
        SceUInt32 uiBufferByte
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (must be called in an interrupt-enabled state)

Not multithread safe

Arguments

pucBufferAddr	Starting address of input data
uiBufferByte	Input buffer size

Return Value

Value	Meaning
>= 0	ATRAC™ ID (An ATRAC™ ID was successfully obtained)
< 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function gets an ATRAC™ ID and associates an input buffer with the ATRAC™ ID. Data from the beginning of the file should be read into the input buffer.
Internally, memory required by libatrac3plus is initialized, the buffer that was configured as *pucBufferAddr* is set, the header of the ATRAC3™/ATRAC3plus™ data that was read into *pucBufferAddr* is analyzed, and an ATRAC™ ID for the corresponding data type is obtained.
This function accepts stereo and monaural data as input data. If the decoding function **sceAtracDecodeData**() is called after initialization is performed by this function, the data will always be output as two channels.

Since part of this function that gets the ATRAC™ ID is multithread safe, duplicate ATRAC™ IDs will not be allocated even if this function is called from multiple threads. However, since other parts of the function are not multithread safe, operation is not guaranteed if decoding is performed from another thread with the same ATRAC™ ID as the argument.

Example

```
/* Get an ATRAC™ ID and associate it with an input buffer */
atracID = sceAtracSetDataAndGetID(buf, databyte);
```

Notes

If data is not read correctly into the input buffer, up to *uiBufferByte* + 2 bytes from *pucBufferAddr* may be accessed.
Therefore, make sure that the 2 bytes after the final address of the read buffer are located in an accessible area.

sceAtracSetAA3DataAndGetID

Set initial data and allocate ATRAC™ ID

Syntax

```
#include <libatrac3plus.h>
#include <libatrac3plus_aa3.h>

SceInt32 sceAtracSetAA3DataAndGetID(
        SceUChar8 *pucBufferAddr,
        SceUInt32 uiBufferByte,
        SceUInt32 uiFileLength,
        SceUInt32 *puiMetadataOffset
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (must be called in an interrupt-enabled state)

Not multithread safe

Arguments

pucBufferAddr	Input data starting address
uiBufferByte	Input buffer size
uiFileLength	File length to be input
**puiMetadataOffset*	Metadata offset position

Return Value

Value	Meaning
>= 0	ATRAC™ ID (An ATRAC™ ID was successfully obtained)
< 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function gets an ATRAC™ ID and associates an input buffer with the ATRAC™ ID. Data from the beginning of the file should be read into the input buffer.
Internally, memory required by libatrac3plus is initialized, the buffer that was configured as *pucBufferAddr* is set, the header of the AA3 data that was read into *pucBufferAddr* is analyzed, and an ATRAC™ ID for the corresponding data type is obtained.
This function accepts stereo data as input data. If the decoding function **sceAtracDecodeData**() is called after initialization is performed by this function, the data will always be output as two channels.
If the data required for playback is not found within the specified input buffer *puiBufferAddr* (the metadata part is too large), this function returns an error and the size of the metadata (in bytes) is stored in *puiMetadataOffset*.
Since part of this function that gets the ATRAC™ ID is multithread safe, duplicate ATRAC™ IDs will not be allocated even if this function is called from multiple threads. However, since other parts of the function are not multithread safe, operation is not guaranteed if decoding is performed from another thread with the same ATRAC™ ID as the argument.

Example

```
/* Get an ATRAC™ ID and associate it with an input buffer */
atracID = sceAtracSetAA3DataAndGetID(buf, databyte, filelength, &metasize);
```

Notes

If data is not read correctly into the input buffer, up to *uiBufferByte* + 2 bytes from *pucBufferAddr* may be accessed.
Therefore, make sure that the 2 bytes after the final address of the read buffer are located in an accessible area.

sceAtracSetMOutDataAndGetID

Set initial data and allocate ATRAC™ ID (monaural output)

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracSetMOutDataAndGetID(
        SceUChar8 *pucBufferAddr,
        SceUInt32 uiBufferByte
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (must be called in an interrupt-enabled state)

Not multithread safe

Arguments

pucBufferAddr	Starting address of input data
uiBufferByte	Input buffer size

Return Value

Value	Meaning
>= 0	ATRAC™ ID (An ATRAC™ ID was successfully obtained)
< 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function gets an ATRAC™ ID and associates it with an input buffer Data from the beginning of the file should be read into the input buffer.
This function accepts only monaural data as input data. If the decoding function **sceAtracDecodeData**() is called after initialization is performed by this function, the data will always be output as one channel.
Internally, memory required by libatrac3plus is initialized, the buffer that was configured as *pucBufferAddr* is set, the header of the ATRAC3™/ATRAC3plus™ data that was read into *pucBufferAddr* is analyzed, and an ATRAC™ ID for the corresponding data type is obtained.

Since part of this function that gets the ATRAC™ ID is multithread safe, duplicate ATRAC™ IDs will not be allocated even if this function is called from multiple threads. However, since other parts of the function are not multithread safe, operation is not guaranteed if decoding is performed from another thread with the same ATRAC™ ID as the argument.

Example

```
/* Get an ATRAC™ ID and associate it with an input buffer */
atracID = sceAtracSetMOutDataAndGetID(buf, databyte);
```

Notes

If data is not read correctly into the input buffer, up to *uiBufferByte* + 2 bytes from *pucBufferAddr* may be accessed.
Therefore, make sure that the 2 bytes after the final address of the read buffer are located in an accessible area.

sceAtracSetHalfwayBufferAndGetID

Set initial data and allocate ATRAC™ ID

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracSetHalfwayBufferAndGetID(
        SceUChar8 *pucBufferAddr,
        SceUInt32 uiReadByte
        SceUInt32 uiBufferByte
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (must be called in an interrupt-enabled state)

Not multithread safe

Arguments

pucBufferAddr	Starting address of input data
uiReadByte	Size of valid data from beginning of input data
uiBufferByte	Input buffer size

Return Value

Value	Meaning
>= 0	ATRAC™ ID (An ATRAC™ ID was successfully obtained)
< 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function gets an ATRAC™ ID and associates it with an input buffer. Data from the beginning of the file should be read into the input buffer.
This function accepts stereo and monaural data as input data. If the decoding function **sceAtracDecodeData**() is called after initialization is performed by this function, the data will always be output as two channels.
Internally, memory required by libatrac3plus is initialized, the buffer that was configured as *pucBufferAddr* is set, the header of the ATRAC3™/ATRAC3plus™ data that was read into *pucBufferAddr* is analyzed, and an ATRAC™ ID for the corresponding data type is obtained.
This function differs from **sceAtracSetDataAndGetID** () in that the amount of data that is read into the buffer can be less than the buffer size. Therefore, the time until sound is first produced can be shortened since the amount of data that must be read up front can be reduced.

Since part of this function that gets the ATRAC™ ID is multithread safe, duplicate ATRAC™ IDs will not be allocated even if this function is called from multiple threads. However, since other parts of the function are not multithread safe, operation is not guaranteed if decoding is performed from another thread with the same ATRAC™ ID as the argument.

Example

```
/* Get an ATRAC™ ID and associate it with an input buffer */
atracID = sceAtracSetHalfwayBufferAndGetID(buf, readbyte,bufsize);
```

Notes

If data is not read correctly into the input buffer, up to *uiBufferByte* + 2 bytes from *pucBufferAddr* may be accessed.
Therefore, make sure that the 2 bytes after the final address of the read buffer are located in an accessible area.

sceAtracSetAA3HalfwayBufferAndGetID

Set initial data and allocate ATRAC™ ID

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracSetAA3HalfwayBufferAndGetID(
        SceUChar8 *pucBufferAddr,
        SceUInt32 uiReadByte
        SceUInt32 uiBufferByte,
        SceUInt32 uiFileLength,
        SceUInt32 *puiMetadataOffset
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (must be called in an interrupt-enabled state)

Not multithread safe

Arguments

pucBufferAddr	Input data starting address
uiReadByte	Valid data size from beginning of input data
uiBufferByte	Input buffer size
uiFileLength	File size to be input
**puiMetadataOffset*	Metadata offset position

Return Value

Value	Meaning
>= 0	ATRAC™ ID (An ATRAC™ ID was successfully obtained)
< 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function gets an ATRAC™ ID and associates an input buffer with the ATRAC™ ID. Data from the beginning of the file should be read into the input buffer.
This function accepts stereo data as input data. If the decoding function **sceAtracDecodeData**() is called after initialization is performed by this function, the data will always be output as two channels.
If the data required for playback is not found within the specified input buffer *puiBufferAddr* (the metadata part is too large), this function returns an error and the size of the metadata (in bytes) is stored in *puiMetadataOffset*.
This function differs from **sceAtracSetAA3DataAndGetID**() in that the data size that is read into the buffer can be less than the buffer size. As a result, since the amount of data that must be read first can be reduced, the time until sound is first produced can be shortened.
Since part of this function that gets the ATRAC™ ID is multithread safe, duplicate ATRAC™ IDs will not be allocated even if this function is called from multiple threads. However, since other parts of the function are not multithread safe, operation is not guaranteed if decoding is performed from another thread with the same ATRAC™ ID as the argument.

Example

```
/* Get an ATRAC™ ID and associate it with an input buffer */
atracID = sceAtracSetAA3HalfwayBufferAndGetID(buf,
readbyte,bufsize,filelength,&metasize);
```

Notes

If data is not read correctly into the input buffer, up to *uiBufferByte* + 2 bytes from *pucBufferAddr* may be accessed.
Therefore, make sure that the 2 bytes after the final address of the read buffer are located in an accessible area.

sceAtracSetMOutHalfwayBufferAndGetID

Set initial data and allocate ATRAC™ ID (monaural output)

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracSetMOutHalfwayBufferAndGetID(
        SceUChar8 *pucBufferAddr,
        SceUInt32 uiReadByte
        SceUInt32 uiBufferByte
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (must be called in an interrupt-enabled state)

Not multithread safe

Arguments

pucBufferAddr	Starting address of input data
uiReadByte	Size of valid data from beginning of input data
uiBufferByte	Input buffer size

Return Value

Value	Meaning
>= 0	ATRAC™ ID (An ATRAC™ ID was successfully obtained)
< 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function gets an ATRAC™ ID and associates it with an input buffer. Data from the beginning of the file should be read into the input buffer.
This function accepts only monaural data as input data. If the decoding function **sceAtracDecodeData**() is called after initialization is performed by this function, the data will always be output as one channel.
Internally, memory required by libatrac3plus is initialized, the buffer that was configured as *pucBufferAddr* is set, the header of the ATRAC3™/ATRAC3plus™ data that was read into *pucBufferAddr* is analyzed, and an ATRAC™ ID for the corresponding data type is obtained.
This function differs from **sceAtracSetMOutDataAndGetID** () in that the amount of data that is read into the buffer can be less than the buffer size. Therefore, the time until sound is first produced can be shortened since the amount of data that must be read up front can be reduced.

Since part of this function that gets the ATRAC™ ID is multithread safe, duplicate ATRAC™ IDs will not be allocated even if this function is called from multiple threads. However, since other parts of the function are not multithread safe, operation is not guaranteed if decoding is performed from another thread with the same ATRAC™ ID as the argument.

Example

```
/* Get an ATRAC™ ID and associate it with an input buffer */
atracID = sceAtracSetMOutHalfwayBufferAndGetID(buf, readbyte,bufsize);
```

Notes

If data is not read correctly into the input buffer, up to *uiBufferByte* + 2 bytes from *pucBufferAddr* may be accessed.
Therefore, make sure that the 2 bytes after the final address of the read buffer are located in an accessible area.

Decode Function

sceAtracDecodeData

Decode data

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracDecodeData(
        SceInt32 atracID,
        SceShort16 *psOutAddr,
        SceUInt32 *puiSamples,
        SceUInt32 *puiFinishflag,
        SceInt32 *piRemainframe
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (must be called in an interrupt-enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
psOutAddr	Pointer to the memory area where the result of the decode is written
puiSamples	Pointer to the variable into which the number of samples that were decoded is entered
puiFinishflag	Pointer to the variable into which the flag indicating whether all data was decoded is entered
piRemainframe	Pointer to the variable in which Buffer State Identifier or remaining number of times data can be decoded is set

Return Value

Value	Meaning
0	Normal completion
< 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function performs decoding and outputs audio data. The output data is written in a format having 16 bits per sample to the area indicated by *psOutAddr*. When 2 channels are output, the left channel is interleaved first when it is output (data for the left and right channel data are arranged alternately for each sample).
Users of the library must allocate, using *psOutAddr*, a memory area which is sufficient for the number of output channels and the number of samples. The size of the area required can be calculated by multiplying the value acquired from **sceAtracGetMaxSample**(), and **sceAtracGetNextSample**() by 4 (for 2 channel output), or by 2 (for 1 channel output).
Whether the output data is stereo or monaural depends on which function was used for initialization, not on whether the sound source data is monaural. When monaural data is decoded and stereo output is performed, the same data is output by the left and right channels. The current number of output channels can be checked by using **sceAtracGetOutputChannel**().

Example

```
finishflag = 0;
while (finishflag == 0){
        status = sceAtracDecodeData(atracID, out_buf,
                                    &decodedsample, &finishflag, &remainframe);
        if (status != SCE_ATRAC_SUCCESS){
            break;
        }
        /* Output data */
        /* Additional data */
}
```

Notes

The size of the *psOutAddr* area must be a multiple of 64 bytes and it must be allocated with 64-byte alignment. To perform decoding multiple times collectively, the buffer must be allocated on a 64-byte boundary and its size must be a multiple of 64 bytes even for a large output buffer. However, when the buffer is allocated in this manner, the decoding output start address (*psOutAddr*) only needs to be 2-byte aligned; 64-byte alignment is not required.

When using streaming playback and when playing special data, the second buffer must be set before playback. For details, see the description of **sceAtracSetSecondBuffer**().
This function is not supported for ATRAC™ IDs initialized with **sceAtracLowLevelInitDecoder**().

See Also

Buffer State Identifier
sceAtracSetData()
sceAtracSetHalfwayBuffer()
sceAtracSetDataAndGetID()
sceAtracSetHalfwayBufferAndGetID()
sceAtracGetOutputChannel()

Streaming and Buffer-Related Functions

sceAtracGetStreamDataInfo

Get streaming information

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracGetStreamDataInfo (
        SceInt32 atracID,
        SceUChar8 **ppucWritePointer,
        SceUInt32 *puiWritableByte,
        SceUInt32 *puiReadPosition
        );
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or -enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
ppucWritePointer	Pointer to the variable which stores the value indicating the starting write position in memory
puiWritableByte	Number of bytes that can be written from **ppucWritePointer*
puiReadPosition	Starting read position in the file

Return Value

Value	Meaning
0	Normal completion
< 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function gets information of the data to be read next (the read position in the input file, write position in the buffer, and number of writable bytes) during streaming playback.
Based on the values obtained by this function, additional data should be written to an area starting from **ppucWritePosition* when performing streaming playback.
sceAtracAddStreamData() should be called to report the additional data to the library.
When all data is in the buffer or when the buffer is currently full, **puiWritableByte* will be set to 0.

Example

```
finishflag = 0;
while (finishflag == 0){
        /* Get information */
        sceAtracGetStreamDataInfo(atracID,
                                  &pucWritePointer,
                                  &uiWritableByte,
                                  &uiReadPosition);
        /* Set file read position */
        sceIoLseek(fd, uiReadPosition, SCE_SEEK_SET);

        /* Read data from the file */
        uiAddByte = sceIoRead(fd, pucWritePosition, uiWritableByte);
        /* API for reporting the additional data */
        sceAtracAddStreamData(atracID, uiAddByte);
}
```

Notes

You can also use **sceAtracGetStreamDataInfo**() to get additional information about data for the second buffer.
This function is not supported for ATRAC™ IDs initialized with **sceAtracLowLevelInitDecoder**().

See Also

sceAtracAddStreamData()

sceAtracAddStreamData

Report additional data

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracAddStreamData(
        SceInt32 atracID,
        SceUInt32 uiAddByte
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or -enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
uiAddByte	Number of bytes added to the buffer

Return Value

Value	Meaning
0	Normal completion
< 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function reports to the library of the number of bytes of data that were added to the buffer. When data is added, be sure to call this function.

Example

```
finishflag = 0;
while (finishflag == 0){
        /* Get information */
        sceAtracGetStreamDataInfo(atracID,
                                  &pucWritePointer,
                                  &uiWritableByte,
                                  &uiReadPosition);
        /* Set file read position */
        sceIoLseek(fd, uiReadPosition, SCE_SEEK_SET);

        /* Read data from the file */
        uiAddByte = sceIoRead(fd, pucWritePosition, uiWritableByte);
        /* API for reporting the additional data */
        sceAtracAddStreamData(atracID, uiAddByte);
}
```

Notes

You can also use the **sceAtracAddStreamData**() function when adding more data to the second buffer.

This function is not supported for ATRAC™ IDs initialized with **sceAtracLowLevelInitDecoder**().

See Also

sceAtracGetStreamDataInfo()

sceAtracGetSecondBufferInfo

Get second buffer information

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracGetSecondBufferInfo(
        SceInt32 atracID,
        SceUInt32 *puiPosition,
        SceUInt32 *puiDataByte
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or -enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
puiPosition	Position in the file for reading data into the second buffer
puiDataByte	Number of bytes to the end of the file from *puiPosition*

Return Value

Value	Meaning
0	Normal completion
< 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function is used during streaming playback of ATRAC3™/ATRAC3plus™ data when there is data after the end of a loop. The data beyond the loop should be loaded into the second buffer in advance. For details, refer to the libatrac3plus Overview.

Example

```
ret = sceAtracGetSecondBufferInfo(atracID,
                                  &position,
                                  &filesize);
sceIoLseek(fd, position, SCE_SEEK_SET);

if (!(filesize = sceIoRead(fd, secondBuf, filesize))) {
        printf("file size = %d\n", filesize);
        return -1;
}
ret = sceAtracSetSecondBuffer(atracID,
                              secondBuf,
                              filesize);
```

Notes

The second buffer must be set when performing streaming playback of ATRAC3™/ATRAC3plus™ data which outputs audio after the end of a loop. Note that if **sceAtracDecodeData**() is called without setting the second buffer in this situation, it will return SCE_ATRAC_ERROR_NEED_SECOND_BUFFER.

This function is not supported for ATRAC™ IDs initialized with **sceAtracLowLevelInitDecoder**().

See Also

sceAtracDecodeData(), **sceAtracSetSecondBuffer**()

sceAtracSetSecondBuffer

Set second buffer

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracSetSecondBuffer(
        SceInt32 atracID,
        SceUChar8 *pucSeconddBufferAddr,
        SceUInt32 uiSecondBufferByte
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or -enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
pucSecondBufferAddr	Pointer to the starting point of the Second buffer (a user-allocated area should be specified)
uiSecondBufferByte	Size of second buffer

Return Value

Value	Meaning
0	Normal completion
< 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function is used during streaming playback of ATRAC3™/ATRAC3plus™ data when there is data after the end of a loop. The data beyond the loop should be loaded into the second buffer in advance. For details, refer to the libatrac3plus Overview.

sceAtracGetSecondBufferInfo() can be used to get information about the position in the file where data is to be read into the second buffer, and the number of bytes to read from that position to the end of the file.

Example

```
ret = sceAtracGetSecondBufferInfo(atracID,
                                  &position,
                                  &filesize);
sceIoLseek(fd, position, SCE_SEEK_SET);

if (!(filesize = sceIoRead(fd, secondBuf, filesize)))   {
        printf("file size = %d\n", filesize);
        return -1;
}
ret = sceAtracSetSecondBuffer(atracID,
                              secondBuf,
                              filesize);
```

Notes

The second buffer must be set when performing streaming playback of ATRAC3™/ATRAC3plus™ data which outputs audio after the end of a loop. Note that if **sceAtracDecodeData**() is called without setting the second buffer in this situation, it will return SCE_ATRAC_ERROR_NEED_SECOND_BUFFER.

This function is not supported for ATRAC™ IDs initialized with **sceAtracLowLevelInitDecoder**().

See Also

sceAtracGetSecondBufferInfo()

sceAtracIsSecondBufferNeeded

Check second buffer

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracIsSecondBufferNeeded(
        SceInt32 atracID
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or -enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data

Return Value

Value	Description
0	Second buffer is not necessary
Positive (>0)	Second buffer is necessary
< 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function returns a value indicating whether or not the second buffer is necessary to play the ATRAC3™/ATRAC3plus™ data that has been set.

The second buffer is required to perform streaming playback of ATRAC3™/ATRAC3plus™ data when audio is output after the end of a loop.

Example

```
/* Get data in the second buffer only when the second buffer is necessary */
if (sceAtracIsSecondBufferNeeded(atracID) > 0) {
        ret = sceAtracGetSecondBufferInfo(atracID,
                                          &position,
                                          &filesize);
        sceIoLseek(fd, position, SCE_SEEK_SET);

        if (!(filesize = sceIoRead(fd, secondBuf, filesize)))   {
                printf("file size = %d\n", filesize);
                return -1;
        }
        ret = sceAtracSetSecondBuffer(atracID,
                                      secondBuf,
                                      filesize);
}
```

Notes

The second buffer must be set to perform streaming playback of ATRAC3™/ATRAC3plus ™data when audio is output after the end of a loop.

Since this function can be called to check whether or not the ATRAC3™/ATRAC3plus™ data that is about to be played needs the second buffer, be sure to allocate the second buffer accordingly.

This function is not supported for ATRAC™ IDs initialized with **sceAtracLowLevelInitDecoder**().

See Also

sceAtracGetSecondBufferInfo ()

sceAtracSetSecondBuffer ()

sceAtracGetRemainFrame

Get remaining number of decodes

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracGetRemainFrame(
        SceInt32 atracID,
        SceInt32 *piRemainframe
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or -enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
piRemainframe	Pointer to variable in which Buffer State Identifier or remaining number of times data can be decoded is set

Return Value

Value	Description
0	Normal completion
Other than 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This is a function for getting the state (Buffer State Identifier or remaining number of times data can be decoded) of the buffer that is currently being used.

Example

```
/* Get buffer state */
status = sceAtracGetRemainFrame(atracID, &remainframe);
```

Notes

This function can be used to obtain the state (Buffer State Identifier or remaining number of times data can be decoded) of the buffer that is currently being used. Using this funcion, you can check whether processing to add to the buffer is necessary for the buffer that is currently being used.
This function is not supported for ATRAC™ IDs initialized with **sceAtracLowLevelInitDecoder**().

See Also

Buffer State Identifier
sceAtracSetData()
sceAtracSetHalfwayBuffer()
sceAtracSetDataAndGetID()
sceAtracSetHalfwayBufferAndGetID()

Audio Information Acquisition Functions

sceAtracGetChannel

Get the number of channels of the input data

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracGetChannel(
        SceInt32 atracID,
        SceUInt32 *puiChannel
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or -enabled state)

Multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
puiChannel	Pointer for storing the number of channels

Return Value

Value	Meaning
0	Normal completion
Other than 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function gets the number of recorded channels in the data that is currently set. Using this function, you can determine whether the input data is 1 channel or 2 channels. However, the number of channels of the data that is output to *psOutAddr* also depends on the initialization method and not just on the input data. The number of output channels can be obtained by using the **sceAtracGetOutputChannel**() function.
An initialization function must be called before calling this function.

Example

```
/* Check the number of channels of encoding data*/
status = sceAtracGetChannel(atracID, &channels);
```

Notes

When operating on the same data, this function always returns the same value.

See Also

sceAtracSetData()

sceAtracSetHalfwayBuffer()

sceAtracSetDataAndGetID()

sceAtracSetHalfwayBufferAndGetID()

sceAtracLowLevelInitDecoder()

sceAtracGetOutputChannel()

sceAtracGetOutputChannel

Get the number of output channels

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracGetOutputChannel(
        SceInt32 atracID,
        SceUInt32 *puiOutChannel
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or -enabled state)

Multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
puiOutChannel	Pointer for storing the number of channels

Return Value

Value	Description
0	Normal completion
Other than 0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function gets the number of channels of data that are currently specified.

Using this function, you can determine whether the output data is 1 channel or 2 channels.

An initialization function must be called before calling this function.

Example

```
/*Check the number of output channels */
status = sceAtracGetOutputChannel(atracID, &channels);
```

Notes

When the same data is handled, this function always returns the same value.

See Also

sceAtracSetData()

sceAtracSetHalfwayBuffer()

sceAtracSetDataAndGetID()

sceAtracSetHalfwayBufferAndGetID()

sceAtracLowLevelInitDecoder()

sceAtracGetChannel()

sceAtracGetMaxSample

Get maximum number of output samples

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracGetMaxSample(
        SceInt32 atracID,
        SceUInt32 *puMaxSample
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or -enabled state)

Multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
puiMaxSample	Pointer for storing the maximum number of output samples

Return Value

Value	Meaning
0	Normal completion
<0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function gets the maximum number of samples per channel that are output when **sceAtracDecodeData**() is executed. By quadrupling (2 bytes (16 bits) x 2 channels) this value for 2-channel output or doubling it for 1-channel output, you can find the size in bytes of the output area that should be provided when calling **sceAtracDecodeData**().
You must call an initialization function before calling this function.
Constant values are always returned in **puiMaxSample* depending on the format; for ATRAC3™ it is 1024, and for ATRAC3plus™ it is 2048.

Example

```
/* Check maximum number of samples */
status = sceAtracGetMaxSample (atracID,&maxsample);
```

Notes

When operating on the same data, this function always returns the same value.

See Also

sceAtracSetData()

sceAtracSetHalfwayBuffer()

sceAtracSetDataAndGetID()

sceAtracSetHalfwayBufferAndGetID()

sceAtracLowLevelInitDecoder()

sceAtracGetNextSample

Get the number of output samples in next decoding

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracGetNextSample (
        SceInt32 atracID,
        SceUInt32 *puiNextSample
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or -enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
puiNextSample	Pointer for storing the number of output samples in next decoding

Return Value

Value	Meaning
0	Normal completion
<0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function gets the number of output samples per channel that are to be returned for the next decoding. By quadrupling (2 bytes (16 bits) x 2 channels) this value for 2-channel output or doubling it for 1-channel output, you can find the size in bytes of the output area that should be provided when calling **sceAtracDecodeData**().

You must call an initialization function before calling this function.

Example

```
/* Check the number of output samples for the next decoding */
status = sceAtracGetNextSample (atracID,&nextsample);
```

Notes

The value obtained from **sceAtracGetNextSample**() may be different each time.

This function is not supported for ATRAC™ IDs initialized with **sceAtracLowLevelInitDecoder**().

See Also

sceAtracSetData()

sceAtracSetHalfwayBuffer()

sceAtracSetDataAndGetID()

sceAtracSetHalfwayBufferAndGetID()

sceAtracGetSoundSample

End sample and loop sample retrieval function

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracGetSoundSample(
        SceInt32 atracID,
        SceInt32 *piEndSample,
        SceInt32 *piLoopStartSample,
        SceInt32 *piLoopEndSample,
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or -enabled state)

Multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
piEndSample	Pointer to store the end sample of the music
piLoopStartSample	Pointer to store the loop start sample of the music
piLoopEndSample	Pointer to store the loop end sample of the music

Return Value

Value	Meaning
0	Normal completion
<0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function retrieves the end sample, loop start sample, and loop end sample of the currently specified data.

Note that the initial sample number at which to start the music is counted from 0.

If the currently specified data does not have loop information, set **piLoopStartSample* and **piLoopEndSample* to -1.

Example

```
/* Check number of channels of encoding data */
status = sceAtracGetSoundSample(atracID,
                                &iEndSample,
                                &iLoopStartSample,
                                &iLoopEndSample);
```

Notes

For ATRAC3™/ATRAC3plus™ data for which the number of samples for the music in its entirety is n, the start sample, end sample, loop start sample, and loop end sample are as shown in Figure 1.

The length of the music in its entirety can be found as the **end sample + 1**.

In addition, the number of samples in the loop portion is **loop end position + 1 – loop start position**.

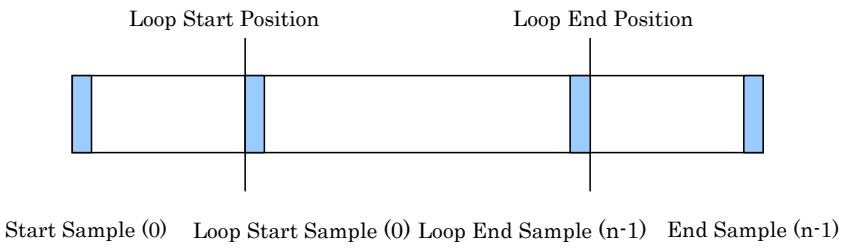

Figure 1 Relationship Between n-Sample Music and Sample Position

This function is not supported for ATRAC™ IDs initialized with **sceAtracLowLevelInitDecoder**().

See Also

sceAtracSetData()

sceAtracSetHalfwayBuffer()

sceAtracSetDataAndGetID()

sceAtracSetHalfwayBufferAndGetID()

sceAtracGetNextDecodePosition

Decoding sample position retrieval function

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracGetNextDecodePosition(
        SceInt32 atracID,
        SceUInt32 *puiSamplePosition
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or -enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
puiSamplePosition	Pointer to store playback position

Return Value

Value	Meaning
0	Normal completion
<0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function finds the decode sample position (the starting position of the sample to be output during the next decoding operation) of the currently specified data.
If all of the data has been decoded, SCE_ATRAC_ERROR_ALLDATA_WAS_DECODED is returned.

Example

```
/* Check number of channels of encoded data */
status = sceAtracGetNextDecodePosition(atracID, &uiSamplePosition);
```

Notes

The sample count starts from 0.
In other words, an n-sample ATRAC3™/ATRAC3plus™ data will occupy sample positions 0 through n-1.
The value which is obtained with this function represents the position of the sample being decoded, and not the position of the sample being played back. To utilize the correct playback position, it is necessary to consider the size and number of buffers to use for output to the audio driver.
This function is not supported for ATRAC™ IDs initialized with **sceAtracLowLevelInitDecoder()**.

See Also

sceAtracSetData()
sceAtracSetHalfwayBuffer()
sceAtracSetDataAndGetID()
sceAtracSetHalfwayBufferAndGetID()

sceAtracGetBitrate

Bit rate retrieval function

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracGetBitrate(
        SceInt32 atracID,
        SceUInt32 *puiBitrate
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or enabled state)

Multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
puiBitrate	Pointer for storing the bit rate

Return Value

Value	Description
0	Successful completion
<0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function finds the bit rate of the currently specified data.

Example

```
/* Check number of channels of encoded data */
status = sceAtracGetBitrate(atracID, &uiBitrate);
```

Notes

This function will always return the same value when handling the same data.

See Also

sceAtracSetData()

sceAtracSetHalfwayBuffer()

sceAtracSetDataAndGetID()

sceAtracSetHalfwayBufferAndGetID()

sceAtracLowLevelInitDecoder()

sceAtracGetLoopStatus

Loop iteration count and loop status retrieval function

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracGetLoopStatus(
        SceInt32 atracID,
        SceInt32 *piLoopNum,
        SceUInt32 *puiLoopStatus
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
piLoopNum	Pointer for storing the number of loops remaining in the specified ATRAC3™/ATRAC3plus™ data
puiLoopStatus	Pointer for storing the loop status (playback position) of the specified ATRAC3™/ATRAC3plus™ data

Return Value

Value	Description
0	Successful completion
<0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function finds the number of loops remaining and the loop status (playback position) of the specified ATRAC3™/ATRAC3plus™ data.

The following values can be set in **piLoopNum*.

Value	Description
0	No loop playback
> 0	Perform loop playback **piLoopNum* times
< 0	Perform continuous loop playback

The values set in **piLoopStatus* are as follows.

Value	Description
1	Playing back data prior to the start of the loop, or playing back data in the loop, for data with loop information.
0	Playing back data without loop information or playing the portion after the loop ends for data with loop information.

Example

```
/* Get loop information */
status = sceAtracGetLoopStatus(atracID, &num, &stat);
```

Notes

sceAtracGetLoopStatus() may return a different value each time.

Note that, when referencing this function in order to invoke **sceAtracSetLoopNum**(), the value returned may vary depending upon the timing with respect to **sceAtracDecodeData**().

This function is not supported for the playback of AA3 files.

This function is not supported for ATRAC™ IDs initialized with **sceAtracLowLevelInitDecoder**().

See Also

sceAtracSetLoopNum()

Loop Count Setting Function

sceAtracSetLoopNum

Remaining loop count setting function

Syntax

```
#include <libatrac3plus.h>
int sceAtracSetLoopNum (
        SceInt32 atracID,
        SceInt32 iLoopNum
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
iLoopNum	Remaining loop count to reset in the specified ATRAC3™/ATRAC3plus™ data

Return Value

Value	Description
0	Successful completion
<0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function resets the remaining loop count.

The values which can be set for *iLoopNum* are as follows.

Value	Description
0	No loop playback
> 0	Perform loop playback *iLoopNum* times
< 0	Perform continuous loop playback

However, the number of loops can be changed only when the following two conditions are satisfied.

- The specified ATRAC3™/ATRAC3plus™ data has set the loop starting position and loop ending position.
- The part to be decoded next is either before the start of the loop or is inside the loop.

You can find what part of the data will be decoded next by using **sceAtracGetLoopStatus**().

The loop count is set to 0 (no loop playback) immediately after an initialization function (**sceAtracSetData**(), **sceAtracSetHalfwayBuffer**(), **sceAtracSetDataAndGetID**(), or **sceAtracSetHalfwayBufferAndGetID**() and their monaural output versions) or the playback position modification function (**sceAtracResetPlayPosition**()) is called.

If you won't know the number of times the loop should be played until some later time, you can change the loop count without using **sceAtracGetLoopStatus()** by initially setting the loop to play indefinitely, then reducing the number of iterations later.

Example

```
/* Set loop count to 10 */
status = sceAtracSetLoopNum(atracID, 10);
```

Notes

This function is not supported for AA3 file playback.

This function is not supported for ATRAC™ IDs initialized with **sceAtracLowLevelInitDecoder**().

See Also

sceAtracGetLoopStatus()

Playback Position Resetting Functions

sceAtracGetBufferInfoForResetting

Function to retrieve buffer control information when modifying playback position

Syntax

```
#include <libatrac3plus.h>
int sceAtracGetBufferInfoForResetting(
        SceInt32 atracID,
        SceUInt32 uiSample,
        SceAtracBufferInfo *pBufferInfo
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
uiSample	Playback sample position to reset
pBufferInfo	Pointer to structure for storing buffer information when resetting playback position

Return Value

Value	Meaning
0	Normal completion
<0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function retrieves the necessary information related to buffer control when resetting the playback position.

By calling this function, the following information necessary for resetting the playback position can be retrieved.

- Start address for writing to buffer
- Maximum number of bytes which can be written to buffer
- Minimum number of bytes which must be written to buffer
- Location for reading data to be written to buffer

Example

```
/* Get the buffer control information necessary when changing playback position and
store in uiResetSample*/

status = sceAtracGetBufferInfoForResetting(atracID, uiResetSample,&bufferInfo);
```

Notes

The sample count specified in *uiSample* starts from 0.

In addition, the values which can be set can be obtained using the **sceAtracGetSoundSample()** function.

This function is not supported for AA3 file playback.

This function is not supported for ATRAC™ IDs initialized with **sceAtracLowLevelInitDecoder**().

See Also

SceAtracBufferInfo structure

sceAtracGetSoundSample()

sceAtracResetPlayPosition()

sceAtracResetPlayPosition

Playback position resetting function

Syntax

```
#include <libatrac3plus.h>
int sceAtracResetPlayPosition(
        SceInt32 atracID,
        SceUInt32 uiSample,
        SceUInt32 uiWriteByteFirstBuf,
        SceUInt32 uiWriteByteSecondBuf
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
uiSample	Playback sample position to reset
uiWriteByteFirstBuf	Number of bytes written to the first buffer
uiWriteByteSecondBuf	Number of bytes written to the second buffer

Return Value

Value	Meaning
0	Normal completion
<0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function resets the playback position.

To use this function during streaming playback, first call **sceAtracGetBufferInfoForResetting**() to obtain buffer control information. Then, assign that information to the *uiWriteByteFirstBuf* and *uiWriteByteSecondBuf* arguments so that it can be used to reset the buffer data

When performing on-memory playback, you do not need to reset the buffer data because all of the data is in the buffer. In this case, be sure to set 0 for both of the arguments mentioned above.

When this function is called, the loop count is reset to 0. To maintain the loop count, use the **sceAtracGetLoopStatus**() and **sceAtracSetLoopNum**() functions to get and reset the loop count before and after calling this function.

Example

```
/* Change the playback position to uiResetSample */
status = sceAtracResetPlayPosition(atracID,
                                   uiResetSample,
                                   uiWriteByteFirstBuf,
                                   uiWriteByteSecondBuf);
```

Notes

The sample count specified in *uiSample* starts from 0.

In addition, the values which can be set can be obtained using the **sceAtracGetSoundSample**() function.

This function is not supported for AA3 file playback.

This function is not supported for ATRAC™ IDs initialized with **sceAtracLowLevelInitDecoder**().

See Also

SceAtracBufferInfo structure

sceAtracGetSoundSample()

sceAtracGetBufferInfoForResetting()

Low-Level Functions

sceAtracLowLevelInitDecoder

Initialization function for low-level API

Syntax

```
#include <libatrac3plus.h>
SceInt32 sceAtracLowLevelInitDecoder(
        SceInt32 atracID,
        SceAtracLowLevelParam *pLowLevelParam
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or -enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
pLowLevelParam	Pointer to initialization parameter structure

Return Value

Value	Description
0	Normal completion
<0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function initializes the low-level API of the libatrac3plus library.

The low-level API feature decodes one frame and does not perform processing such as buffer management.

Example

```
SceAtracLowLevelParam lowLevelParam;

/* Set parameters for ATRAC3™ 132 kbps. */
lowLevelParam.nchin          = 2;
lowLevelParam.nchout         = 2;
lowLevelParam.nBytesFrame    = 384;

status = sceAtracLowLevelInitDecoder (atracID,
                                      &lowLevelParam);
```

Notes

ATRAC™ IDs that were initialized with **sceAtracLowLevelInitDecoder**() cannot be used when calling the following functions.

sceAtracDecodeData(),

sceAtracGetStreamDataInfo(),

sceAtracAddStreamData(),

sceAtracGetSecondBufferInfo(),

sceAtracSetSecondBuffer(),

sceAtracIsSecondBufferNeeded(),

sceAtracGetRemainFrame(),

sceAtracGetNextSample(),

sceAtracGetSoundSample(),

sceAtracGetNextDecodePosition(),

sceAtracGetLoopStatus(),

sceAtracSetLoopNum(),

sceAtracGetBufferInfoForResetting(),

sceAtracResetPlayPosition()

See Also

sceAtracLowLevelDecode()

sceAtracLowLevelDecode

Decode function for low-level API

Syntax

```
#include <libatrac3plus.h>
SceInt32 sceAtracLowLevelDecode(
        SceInt32 atracID,
        const SceUChar8 *pucESBuf,
        SceUInt32 *puiUsedSize,
        SceShort16 *psOutAddr,
        SceUInt32 *puiOutSize
);
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or -enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
pucESBuf	Pointer to input data buffer
puiUsedSize	Pointer to variable for storing size of input data that was used
psOutAddr	Starting address of output buffer
puiOutSize	Pointer to variable for storing output size

Return Value

Value	Description
0	Normal completion
<0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function performs decoding and outputs audio data. The output data is written in a format having 16 bits per sample to the area indicated by *psOutAddr*. When 2 channels are output, the left channel data is interleaved first when it is output (data for the left and right channel data are arranged alternately for each sample).
Users of the library must allocate, using *psOutAddr*, a memory area that is sufficient for the number of output channels and the number of samples. The size of the area required can be calculated by multiplying the value obtained from **sceAtracGetMaxSample**() by 4 (for 2 channel output), or by 2 (for 1 channel output).
Whether the output data is stereo or monaural is determined by the value of the *nchout* member of the **SceAtracLowLevelParam** structure during initialization, not on whether the sound source data is monaural.
When monaural data is decoded and stereo output is performed, the same data is output by the left and right channels. The current number of output channels can be checked by using **sceAtracGetOutputChannel**().

Example

```
status = sceAtracDecodeData(atracID,
                            pucEsBuf,
                            &uiUsedSize,
                            psOutAddr,
                            &uiOutSize);
```

Notes

The size of the *psOutAddr* area must be a multiple of 64 bytes and it must be allocated with 64-byte alignment. To perform decoding multiple times collectively, the output buffer must be allocated on a 64-byte boundary and its size must be a multiple of 64 bytes even for a large output buffer. However, when the buffer is allocated in this manner, the decoding output starting address (*psOutAddr*) only needs to be 2-byte aligned; 64-byte alignment is not required.

See Also

sceAtracLowLevelInitDecoder()

Detailed Error Acquisition Function

sceAtracGetInternalErrorInfo

Get detailed error information

Syntax

```
#include <libatrac3plus.h>

SceInt32 sceAtracGetInternalErrorInfo(
        SceInt32 atracID,
        SceInt32 *piResult
)
```

Calling Conditions

Cannot be called from an interrupt handler

Can be called from a thread (does not depend on interrupt-disabled or -enabled state)

Not multithread safe

Arguments

atracID	ID for managing the decoding of ATRAC3™/ATRAC3plus™ data
**piResult*	Pointer for storing internal library error information **piResult* = 0 (no error) **piResult* ≠ 0 (error occurred)

Return Value

Value	Meaning
0	Normal completion
<0	Error

For return values when an error occurs, see the List of Error Codes at the end of this document.

Description

This function checks detailed information when a libatrac3plus function returns a library internal error (SCE_ATRAC_ERROR_API_FAIL). It is assumed that this function will mainly be used for debugging. Although you can execute this function on an actual PSP™, it is not recommended.

Notes

Basically, the error code SCE_ATRAC_ERROR_API_FAIL is only returned when encoded data that was provided to libatrac3plus contained an error. The following two situations can be considered as reasons for this to occur.

(1) The error was inserted in the encoded data from the beginning.
(2) The data was not read correctly by an application, or the data was corrupted by some program.

To verify (1), try to play the same data on the development host PC. Use at3tool to decode the relevant data and check whether the error occurs. If an error occurs here, the problem is in the data itself. If this phenomenon occurs despite the fact that the relevant data had been created with at3tool, notify SCE since it may be caused by an at3tool bug.

Once (1) has been confirmed not to be the cause, then (2) should be considered. Verify that the intended data was read by simply reading the data (there is no defect in the media, etc.), that another thread did not perform a write that exceeded the buffer size, that the data has not been overwritten because of a cache incoherency when the data was DMA transferred, etc.

Constant Definitions

Codec Identifier

Codec specification identifier

Definition

Identifier	Value	Audio Codec Type
SCE_ATRAC_AT3PLUS	0x00001000	ATRAC3plus™
SCE_ATRAC_AT3	0x00001001	ATRAC3™

Description

This identifier is used for specifying the codec when calling **sceAtracGetAtracID**() and so on.

See Also

sceAtracGetAtracID ()

Buffer State Identifier

Buffer state identifier

Definition

Identifier	Value	State
SCE_ATRAC_ALLDATA_IS_ON_MEMORY	-1	On-memory playback is currently being performed.
SCE_ATRAC_NONLOOP_STREAM_DATA_IS_ON_MEMORY	-2	Streaming playback is now being performed for data with no loop information or data after the loop, and all data has been loaded in the buffer.
SCE_ATRAC_LOOP_STREAM_DATA_IS_ON_MEMORY	-3	Data that contains a loop is being played, and the data required for the number of loops that is currently specified has been loaded in the buffer that is currently being used.

Description

This is the buffer information identifier that is set in **piRemainframe* when **sceAtracDecodeData**() or **sceAtracGetRemainFrame**() is called. If the buffer information identifier state is not satisfied, these variables are set with the number of remaining decode operations for the buffer.

When SCE_ATRAC_ALLDATA_IS_ON_MEMORY is set, since on-memory playback is being performed, it is not necessary to add data to the buffer.

When SCE_ATRAC_NONLOOP_STREAM_DATA_IS_ON_MEMORY is set, since all data beginning with and after the position that is currently being decoded during streaming playback has been loaded in the buffer, it is not necessary to add data to the buffer.

When SCE_ATRAC_LOOP_STREAM_DATA_IS_ON_MEMORY is set, the data for the specified number of loops has been loaded in the buffer while streaming playback is being performed for data that includes the loop. Unless the loop count is changed, it is not necesssary to add data to the buffer that is currently being used.
Note that you can still add data to the buffer even if SCE_ATRAC_LOOP_STREAM_DATA_IS_ON_MEMORY is set.

[Note]
When a second buffer is used, care is required when SCE_ATRAC_LOOP_STREAM_DATA_IS_ON_MEMORY is set.
SCE_ATRAC_LOOP_STREAM_DATA_IS_ON_MEMORY is an identifier indicating that data does not need to be added to the loop data buffer that is currently being used (first buffer). When the loop count is changed, and playback of the first buffer ends and the buffer to be used is switched to the second buffer, it may be necessary at times to add data to the second buffer.

See Also

sceAtracDecodeData ()
sceAtracGetRemainFrame ()

Error Codes

List of Error Codes

Error codes returned by libatrac3plus

Macro	Value	Description
SCE_ATRAC_SUCCESS	0x00000000	Processing was successful
SCE_ATRAC_ERROR_PARAM_FAIL	0x80630001	Invalid input parameter
SCE_ATRAC_ERROR_API_FAIL	0x80630002	An internal error occurred (error occurred when codec library was called)
SCE_ATRAC_ERROR_NO_ATRACID	0x80630003	No available ATRAC™ ID
SCE_ATRAC_ERROR_BAD_CODECTYPE	0x80630004	The value of argument *uiCodecType* was invalid when **sceAtracGetAtracID**() was executed
SCE_ATRAC_ERROR_BAD_ATRACID	0x80630005	Invalid ATRAC™ ID
SCE_ATRAC_ERROR_UNKNOWN_FORMAT	0x80630006	Specified data is not ATRAC3™/ATRAC3plus™ data
SCE_ATRAC_ERROR_UNMATCH_FORMAT	0x80630007	Specified data did not match *uiCodecType* when **sceAtracGetAtracID**() was executed
SCE_ATRAC_ERROR_BAD_DATA	0x80630008	There was a problem with the specified data
SCE_ATRAC_ERROR_ALLDATA_IS_ONMEMORY	0x80630009	When **sceAtracAddStreamData**() was executed, an attempt was made to add data, even though all the data was already in the buffer
SCE_ATRAC_ERROR_UNSET_DATA	0x80630010	No input buffer has been set
SCE_ATRAC_ERROR_READSIZE_IS_TOO_SMALL	0x80630011	The data size that was entered is too small to be decoded
SCE_ATRAC_ERROR_NEED_SECOND_BUFFER	0x80630012	The second buffer is required for special data, but it has not been set.

Macro	Value	Description
SCE_ATRAC_ERROR_READSIZE_OVER_BUFFER	0x80630013	The specified read size is larger than the size of the buffer prepared when **sceAtracSetHalfwayBuffer**( ) or **sceAtracSetHalfwayBuffer AndGetID**() are executed
SCE_ATRAC_ERROR_NOT_ALIGNED_ADDRESS	0x80630014	Alignment of output buffer pointer (*psOutAddr*) was not correct when **sceAtracDecodeData**() was executed
SCE_ATRAC_ERROR_BAD_SAMPLE	0x80630015	The value of *uiSample* that you attempted to reset is invalid.
SCE_ATRAC_ERROR_WRITEBYTE_FIRST_BUFFER	0x80630016	When **sceAtracResetPlayPosition** () was executed, the value of *uiWriteByteFirstBuf* was invalid.
SCE_ATRAC_ERROR_WRITEBYTE_SECOND_BUFFER	0x80630017	When sceAtracResetPlayPosition( ) was executed, the value of *uiWriteByteSecondBuf* was invalid.
SCE_ATRAC_ERROR_ADD_DATA_IS_TOO_BIG	0x80630018	When **sceAtracAddStreamData**() was executed, the added data size (*uiAddByte)* was invalid.
SCE_ATRAC_ERROR_NOT_MONO_DATA	0x80630019	The data that was input is not monaural data
SCE_ATRAC_ERROR_UNSET_PARAM	0x80630021	When **sceAtracSetLoopNum**() was executed, the value could not be changed
SCE_ATRAC_ERROR_NONEED_SECOND_BUFFER	0x80630022	The second buffer was not needed to play the ATRAC3™/ATRAC3plus™ data that has been set
SCE_ATRAC_ERROR_NODATA_IN_BUFFER	0x80630023	When **sceAtracDecodeData**() was executed, decoding could not be performed because there was no data in the buffer data in the buffer
SCE_ATRAC_ERROR_ALLDATA_WAS_DECODED	0x80630024	All the data has been decoded

Macro	Value	Description
SCE_ATRAC_ERROR_LOWLEVEL_INIT_ID	0x80630031	ATRAC™ ID was initialized with **sceAtracLowLevelInitDecoder()**
SCE_ATRAC_ERROR_NOT_LOWLEVEL_INIT_ID	0x80630032	ATRAC™ ID was not initialized with **sceAtracLowLevelInitDecoder()**
SCE_ATRAC_ERROR_USED_BY_SAS	0x80630040	An ATRAC™ ID used exclusively for SAS cannot be used
SCE_ATRAC_ERROR_AA3_UNKNOWN_FORMAT	0x80631001	Unknown data format
SCE_ATRAC_ERROR_AA3_VERSION_MISMATCH	0x80631002	Incorrect file version
SCE_ATRAC_ERROR_AA3_ILLEGAL_SYNTAX	0x80631003	Invalid syntax
SCE_ATRAC_ERROR_AA3_BUFFER_SHORTAGE	0x80631004	Data required for playback not found in input buffer
SCE_ATRAC_ERROR_AA3_UNSUPPORTED	0x80631005	Unsupported data